

Migration Workshop Hands-on Lab

Introduction	2
What we'll cover in the exercises	2
Before you begin	2
JMS Selectors vs. NATS Subjects	3
Hands-on exercises	4
Solace VPNs vs NATS Accounts	6
Hands-on exercises	6
Solace Partitioned queues vs. NATS Partitioned Consumer groups	11
Hands-on exercises	12
Static	12
Elastic	14
Leaf Node Topologies	18
Hands-on exercises	19

Introduction

Explore how to implement familiar JMS and Solace messaging patterns with NATS. In this workshop, you will filter orders, control access between accounts, distribute work across consumers, and store messages at the edge for later delivery to a hub.

You will run the NATS examples only. You do not need a running JMS or Solace environment. Familiarity with publishing and subscribing is helpful; each exercise introduces the NATS concepts it uses.

What we'll cover in the exercises

1. JMS Selectors to NATS subject filtering
2. Solace VPNs to NATS Accounts
3. Solace Partitioned Queues to NATS Partitioned Consumer groups
4. NATS Leaf Node topologies

Before you begin

Install the tools

Tool	Used for	Installation reference
NATS Server	Running a local messaging server	NATS downloads
NATS CLI	Publishing, subscribing, and managing JetStream	NATS CLI
Go	Building the consumer-group CLI, if a binary is not supplied	Install Go
cg	Managing partitioned consumer groups	Consumer-group CLI source

Tool	Used for	Installation reference
Docker with Docker Compose	Running the leaf-node and hub topology	Install Docker

In order to build `cg`, in that directory run:

```
go build -o cg
```

In order to work with the NATS CLI and your local NATS server, ensure you have a NATS context defined and selected:

```
nats context save local --server nats://localhost:4222 --select
```

Open the lab directory

Open the supplied `MigrationWorkshop` folder. It contains:

```
MigrationWorkshop/
  Accounts_VPN/
    hub.conf
    acct-hub.conf
    acct-hub1.conf
  PartitionedGroups/
    scripts/
  LeafNodes/
    compose-cluster-leaf.yml
    conf-cluster-leaf-node/
```

Exercise 2 starts `Accounts\_VPN/hub.conf`, which includes the account definitions in `Accounts\_VPN/acct-hub.conf`. You will edit the included file. Exercise 4 uses the supplied Compose file and the configuration files it references.

JMS Selectors vs. NATS Subjects

Here we will explore the filtering capabilities of NATS as compared to JMS Selectors.

JMS Selectors are SQL 92 expressions that are defined with the server at consumer creation time.

Let's assume we have a payload for an order:

```
{
  "orderid": "12345",
  "item": "tag789",
  "price": 320,
  "store": "abc333",
  "city": "SF",
  "state": "CA"
}
```

Here are some examples of selectors:

Selector A: `state="CA" AND city="SF"` - subscribe to all orders where the state is California and the city is San Francisco.

Selector B: `state="CA" AND city="SF" OR city="LA"` - subscribe to all orders that are from California and the orders can come from either Los Angeles or San Francisco.

Selector C `state="CA" AND city="SF" AND price > 300` - subscribe to all orders where the state is California and the City is San Francisco and the order is greater than 300.

With JMS you can subscribe to any destination (topic) that expects Order payloads, and the selector will filter on the payload itself.

In contrast, with NATS the subject/destination taxonomy is used to describe the "direction" of the order, and the filtering is on the subject NOT on the payload.

Assuming the subject structure is: `orders.<state>.<city>.<store>.<value_bucket>`

Hands-on exercises

Start a simple NATS server in a terminal:

```
nats-server -js
```

In another terminal start the you can publish the message:

```
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

The message will be published but it won't be consumed as there are no listeners.

In another terminal, initiate the listeners.

For Selector A (get all the orders from San Francisco):

```
nats sub 'orders.ca.sf.>'
```

```
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

```
nats pub orders.ca.la.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

For selector B (Get all orders from any city in California):

```
nats sub 'orders.ca.*.>'
```

```
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

```
nats pub orders.ca.la.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

For Selector C (Get all orders from San Francisco that are over \$300):

One thing to note is that NATS does not have the ability to do comparison (>,<), so we have to “fuzzify” the distribution and create subject tokens for certain groups of data sets. For example:

low_value=100-200

medium_value = 200-300

high_value= 300+

In the cases below we would have subjects such as : nats sub [orders.ca.sf.abc333.low_value](#), nats sub [orders.ca.sf.abc333.high_value](#), nats sub [orders.ca.sf.abc333.medium_value](#).

So if we wanted to get all the orders above \$300 from across all of California we do this: .

```
nats sub orders.ca.*.*.high_value
```

```
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

If we want to be a bit more narrow in scope, and get only the orders over \$300 from a specific store in San Francisco we would do this:

```
nats sub orders.ca.sf.abc333.high_value
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

If we keep the same publisher but changed the listener to a low_value, then we don't get the message:

```
nats sub orders.ca.*.*.low_value
nats pub orders.ca.sf.abc333.high_value '{"orderid":"12345","item": "tag789","price": 320,"store": "abc333","city": "SF","state": "CA"}'
```

In summary, selectors give you:

- a higher degree of flexibility on the filtering expression,
- and the publisher is agnostic to the filter,

however they rapidly become a bottleneck at scale.

With subject based filtering, you can scale linearly however, with no impact on performance:

- the publisher needs to know where to send the message
- For comparative logic you need to “fuzzify” the value

So you have to do more planning upfront with subject based filters however you get no performance impact as volumes increase.

Solace VPNs vs NATS Accounts

Solace VPNs are conceptually similar to NATS Accounts, however the implementation is quite different.

Solace VPNs are logical abstractions that include a set of ACLs and that use a Client Profile for connectivity. The ACLs include the subject/topic level permissions. Additionally, VPNs are set up per/broker. In a DMR Cluster you would have to create identical VPNs on each node of the cluster.

With NATS things are much simpler. An account has users, and permissions for users are defined within the confines of that account. Accounts are automatically synchronized across all nodes in a cluster or supercluster.

In this lab we will explore some simple operations on accounts and permissions on NATS.

Hands-on exercises

First off we need to start shutdown the previous NAT server instance, and launch a new one, using the default configuration file. In this file we have 2 Accounts and we will explore permissions settings for users in an Account and sharing access across accounts.

```
nats-server -c Accounts_VPN/hub.conf
```

Let's first try to connect to the server without credentials:

```
nats sub test -s nats://localhost:1234 --no-context  
nats: error: nats: Authorization Violation
```

As it can be seen, we are not allowed to as we have been one of the users defined in the account definitions.!

Next let's connect using user_a from Account A.

```
nats sub test -s nats://localhost:1234 --no-context --user user_a --password user_a  
nats: error: nats: permissions violation: Permissions Violation for Subscription to "test"
```

This also fails because user_a has no access to the "test" subject.

Next let's try connect to a subject that the permissions allow for:

```
nats sub "a.public.>" -s nats://localhost:1234 --no-context --user user_a --password user_a
```

This should work !

Now let's publish a message on this subject, in a different terminal !

```
nats pub a.public.stock "trade data" -s nats://localhost:1234 --user user_a --password user_a  
--no-context
```

We can now see that we were able to publish data connected with the right credentials.

Now, let's try something different. Let's try to publish to the subject using user_b from Account B:

```
nats pub a.public.stock "trade data B" -s nats://localhost:1234 --user user_b --password user_b
nats: error: nats: permissions violation: Permissions Violation for Publish to "a.public.stock"
--no-context
```

As it can be seen, user_b has no access to the subject `a.public.stock`.

In order to allow user_b to publish to the respective subject, we need to update the configuration.

In the AccountsVPN/acct-hub.conf file find the segment:

```
ACCT_B {
  jetstream: enable
  .....

  users = [
    { user: user_b, password: "user_b",
      permissions: {
        publish: {
          #allow: ["b.public.>", "a.public.>", "imported.a.public.svc"]
          allow: ["b.public.>", "imported.a.public.svc"]
          #allow: ["b.public.>", "a.public.>", "imported.a.public.svc", "imported.a.stock.private"]
        }
        subscribe: {
          allow: ["b.public.>", "_INBOX.>", "imported.a.public.events.>"]
          .....
        }
      }
    }
  ]
}
```

Uncomment : `#allow: ["b.public.>", "a.public.>", "imported.a.public.svc"]`


```

ACCT_B {
  jetstream: enable
  .....

  users = [
    { user: user_b, password: "user_b",
      permissions: {
        publish: {
          allow: ["b.public.>", "a.public.>", "imported.a.public.svc"]
          allow: ["b.public.>", "imported.a.public.svc"]
          #allow: ["b.public.>", "a.public.>", "imported.a.public.svc", "imported.a.stock.private"]
        }
        subscribe: {
          allow: ["b.public.>", "__INBOX.>", "imported.a.public.events.>"]
        }
      }
    }
  ]
}

```

Reload the config:

Get the NATS Server PID:

```
ps -e|grep nats
```

```
15359 ttys031 0:02.10 nats-server -c Accounts_VPN/hub.conf
```

Issue a reload signal to the NAT server:

```
nats-server --signal reload=15359
```

Now lets republish the message

```
nats pub a.public.stock "trade data B" -s nats://localhost:1234 --user user_b --password user_b --no-context
```

What happens now, is that user_b is able to publish successfully to [a.public.stock](#)., however the message is not received by the listener user_a !

This is because user_b is bound by the security boundaries of Account B !

In order for user_b to be able to send a message to user_a we need to create a connection across the 2 accounts.

In NATS terms “user_a” is a “service” as it receives and handles the message.

We need to make 2 modifications to our configuration file (*Accounts_VPN/hub.conf*).

1) export the “service” from Account A:

```
ACCT_A {  
  jetstream: enable  
  
  users = [  
    { user: user_a, password: "user_a",  
      permissions: {  
        publish: {  
          allow: ["a.public.>", "svc.b.request"]  
        }  
        subscribe: {  
          allow: ["a.public.>", "_INBOX.>", "svc.b.request.reply.>"]  
        }  
      }  
    }  
  ]  
  
  exports = [  
    { service: "a.public.>" }  
  
    # Service export: let ACCT_B call into a.public.svc  
    { service: "a.public.svc" }  
    # Stream export: let ACCT_B subscribe to a.public.events.>  
    { stream: "a.public.events.>" }  
  ]  
}
```

2) import the “service” into Account B:

```

ACCT_B {
  jetstream: enable
  .....

  users = [
    { user: user_b, password: "user_b",
      permissions: {
        publish: {
          #allow: ["b.public.>", "a.public.>", "imported.a.public.svc"]
          .....
          allow: ["b.public.>", "imported.a.public.svc"]
          #allow: ["b.public.>", "a.public.>", "imported.a.public.svc", "imported.a.stock.private"]
          .....
        }
        subscribe: {
          allow: ["b.public.>", "INBOX.>", "imported.a.public.events.>"]
          .....
        }
      }
    }
  ]

  exports = [
    # Service export: let ACCT_A call into b.public.svc
    { service: "b.public.svc" }
  ]

  imports = [
    # Import ACCT_A's service, mapped to a local subject

    { service: { account: ACCT_A, subject: "a.public.>" } }

    #{ service: { account: ACCT_A, subject: "a.public.stock" }, to: "imported.a.stock.private" }
    { service: { account: ACCT_A, subject: "a.public.svc" }, to: "imported.a.public.svc" }
    # Import ACCT_A's stream, mapped under a local prefix
    { stream: { account: ACCT_A, subject: "a.public.events.>" }, to: "imported.a.public.events.>" }
    .....
  ]
}

```

Now that we have changed the configuration to support “account traversals”, we can reload the config:

```
nats-server --signal reload=15359
```

Now we can republish the message:

```
nats pub a.public.stock "trade data B " -s nats://localhost:1234 --user user_b --password user_b --no-context
```

On the user_a subscriber side we now see the message:

```
nats sub "a.public.>" -s nats://localhost:1234 --no-context --user user_a --password user_a
16:09:44 Subscribing on a.public.>
[#1] Received on "a.public.stock"
Nats-Request-Info: {"acc":"ACCT_B","rtt":1058541}
trade data B
```

As you can see, to enable traffic across Accounts, all we had to do is enable importing and exporting of the “Service”.

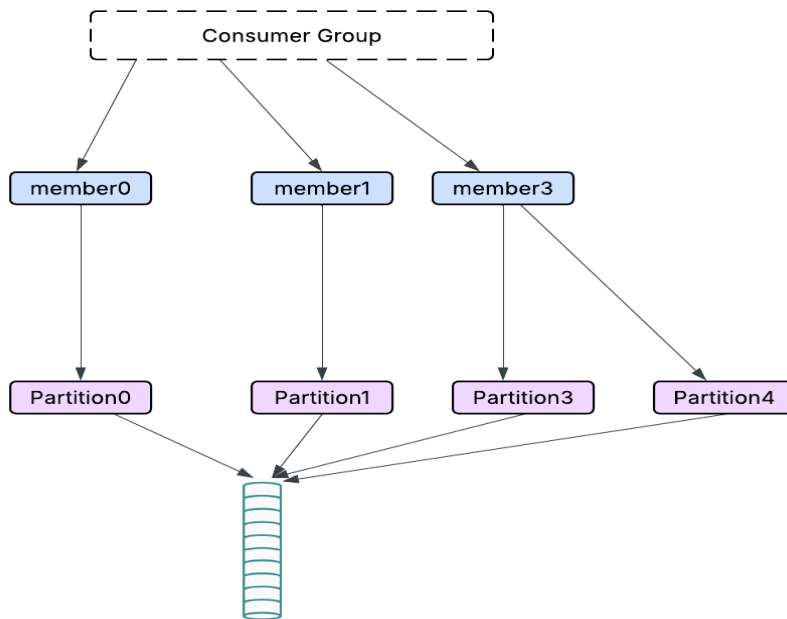
The reverse is also true: if we want to publish from user a (that would make it a “stream”) and would want user_b to receive the message then we would export the “stream” from user_a and importing into user_b.

Because NATS Accounts can live across cluster and superclusters (as long as they have a common Operator), this synchronization happens automatically, and there is no need for any additional configuration. Any change on any account is instantly available across the NATS ecosystem !

Solace Partitioned queues vs. NATS Partitioned Consumer groups

NATS partitions follow a similar model to Kafka. There are however 2 modes of operation:

- Static (kinda like Kafka) - you define up front the number of partitions you want and you create a consumer group with a maximum number of members. The distribution of partitions to members is done by the server. To change the number of members in the consumer group you have to recreate the consumer group.
- Dynamic (kinda like Solace) - you don't need to define the partitions up front. When you define the consumer group with its respective max number of members, the server will implicitly create a number of partitions matching the defined max number of members. You can add and remove members to this group as long as they do not exceed the numbers of members defined in the consumer group.



The General idea is that you can pick any token in a given subject and turn it into a key:

I.e. `orders.<user> => orders.bob, orders.alice, orders.tom` etc. the user becomes the partition key !

Hands-on exercises

In order to test out this scenario we need to create a NATS Context. This is not strictly required however, our CG client is set up to work with a context.

```
nats context save local --server nats://localhost:4222 --description "Local Host"
nats context select local
```

Start a basic nats-server.

```
nats-server -js
```

Static

Let's start by creating a stream ORDERS, with 10 partitions:

```
nats stream add ORDERS_STATIC --subjects="orders.*" --transform-source="orders.*"
--transform-destination="{{partition(10,1)}}.orders.{{wildcard(1)}}" --defaults
```

```
nats stream ls
nats stream info ORDER_STATIC
```

Now, let's create a static consumer group, with max members=10, and actual members m1 and m2:

```
cg static create balanced ORDERS_STATIC s-consumer-grp 10 m1 m2 --filter '>'
cg static info ORDERS_STATIC s-consumer-grp
```

Now, let's add the 2 consumers m1,m2. With 2 instances of each, instance in its own terminal:

```
cg static consume ORDERS_STATIC s-consumer-grp m1 --sleep 25ms
cg static consume ORDERS_STATIC s-consumer-grp m1 --sleep 25ms
cg static consume ORDERS_STATIC s-consumer-grp m2 --sleep 25ms
cg static consume ORDERS_STATIC s-consumer-grp m2 --sleep 25ms
cg static info ORDERS_STATIC s-consumer-grp
```

Let's check the KV store where the consumer group information is being stored:

```
nats kv ls
nats kv ls static-consumer-groups
nats kv get static-consumer-groups ORDERS_STATIC.s-consumer-grp
```

Now let's generate some traffic:

```
nats bench js pub sync orders --multisubject --sleep 10ms --multisubjectmax 100
--stream ORDERS_STATIC
```

And now let's shutdown the first instance of m1; we should see that the standby version kicks in.

You can just Ctrl+C the first m1 consumer.

You can also use an admin interface to “step-down” the consumer:

```
cg static step-down ORDERS_STATIC s-consumer-grp m2
```

Now, if we wanted to add or remove another member for the consumer group, we would have to delete the consumer group and re-created with the respective NEW number of members.

Elastic

If we wanted to:

- 1) Create a stream without indicating upfront the number of partitions
- 2) Create a user group where you can dynamically add/remove members up to the max members quota

We have to use the elastic mode.

To that effect we first create a new stream:

```
nats stream add ORDERS_DYNAMIC --subjects="orders_d.*" --defaults
```

```
nats stream ls
```

Now lets create the consumer group:

```
cg elastic create ORDERS_DYNAMIC d-consumer-grp 10
```

```
filter (empty when done): orders_d.*
```

```
space separated partitioning wildcard indexes: 1
```

```
filter (empty when done):
```

```
nats stream ls
```

Streams					
Name	Description	Created	Messages	Size	Last Message

ORDERS_DYNAMIC	2026-09-28 19:16:04	0	0 B	never
ORDERS_DYNAMIC-d-consumer-grp	2026-09-29 11:59:02	0	0 B	never
ORDERS_STATIC	2026-09-28 16:42:58	13,486	2.2 MiB	17h12m25s

As it can be seen, adding the consumer group created another "internal" stream:
[ORDERS_DYNAMIC-d-consumer-grp](#)

The newly created stream is a WorkQueue stream and it does implement the 10 partitions defined in the consumer group creation:

[nats stream info ORDERS_DYNAMIC-d-consumer-grp](#)

Information for Stream [ORDERS_DYNAMIC-d-consumer-grp](#) created 2026-09-29 11:59:02

Replicas: 1

Storage: File

Options:

Retention: WorkQueue

Acknowledgments: true

Discard Policy: New

Duplicate Window: 0s

Direct Get: true

Allows Batch Publish: false

Allows Counters: false

Allows Msg Delete: true

Allows Per-Message TTL: false

Allows Purge: true

Allows Schedules: false

Allows Rollups: false

Limits:

Maximum Messages: unlimited

Maximum Per Subject: unlimited

Maximum Bytes: unlimited

Maximum Age: unlimited

Maximum Message Size: unlimited

Maximum Consumers: unlimited

Replication:

Sources: [ORDERS_DYNAMIC](#)

Source Information:

Stream Name: [ORDERS_DYNAMIC](#)

Subject Filter and Transform: [orders_d.* to {{Partition\(10,1\)}}:orders_d.{{Wildcard\(0\)}}](#)

Lag: 0

Last Seen: 82ms

State:

Host Version: 2.14.0

Required API Level: 0 hosted at level 4

Messages: 0

Bytes: 0 B

First Sequence: 0

Last Sequence: 0

Active Consumers: 0

So with dynamic partitioning, you:

- You don't need to create the partitions upfront
- You can dynamically add/remove members to the consumer group

Lets create a watcher:

```
watch -n 1 'nats stream info ORDERS_DYNAMIC-d-consumer-grp --json | jq  
"{Messages: .state.messages, Bytes: .state.bytes, FirstSeq: .state.first_seq, LastSeq:  
.state.last_seq}"'
```

Now lets add a member to the consumer group:

```
cg elastic add ORDERS_DYNAMIC d-consumer-grp m1
```

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp  
config: max members=10, max buffered msgs=0, max buffered bytes=0  
filter=orders_d.*, partitioning wildcards [1]  
members: [m1]  
currently active members: []
```

Now lets start a couple of m1 consuming members(one pinned, one standby) - in two separate terminals:

```
cg elastic consume ORDERS_DYNAMIC d-consumer-grp m1  
cg elastic consume ORDERS_DYNAMIC d-consumer-grp m1
```

Now let's do some publishing:

```
nats bench js pub sync orders_d --multisubject --sleep 10ms --multisubjectmax 100  
--stream ORDERS_DYNAMIC
```

Observe the message rate increasing on the Watcher and the consumer.

Let's look at the consumer group membership:

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp
```

```
config: max members=10, max buffered msgs=0, max buffered bytes=0
filter=orders_d.*, partitioning wildcards [1]
members: [m1]
currently active members: [m1]
```

Now we can see on the Watcher that messages are accumulating as the consumer isn't draining the stream fast enough. Lets add a couple of additional consumers:

```
cg elastic add ORDERS_DYNAMIC d-consumer-grp m2 m3
```

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp
config: max members=10, max buffered msgs=0, max buffered bytes=0
filter=orders_d.*, partitioning wildcards [1]
members: [m1 m2 m3]
currently active members: [m1]
```

You can see that the m2,m3 members were added but we still only have 1 active member: m1.

```
cg elastic consume ORDERS_DYNAMIC d-consumer-grp m2
cg elastic consume ORDERS_DYNAMIC d-consumer-grp m3
```

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp
config: max members=10, max buffered msgs=0, max buffered bytes=0
filter=orders_d.*, partitioning wildcards [1]
members: [m1 m2 m3]
currently active members: [m1 m2 m3]
```

Lets now stop one of the consumers:

```
cg elastic step-down ORDERS_DYNAMIC d-consumer-grp m1
cg elastic drop ORDERS_DYNAMIC d-consumer-grp m1
```

Now we only have m2,m3 running.

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp
config: max members=10, max buffered msgs=0, max buffered bytes=0
filter=orders_d.*, partitioning wildcards [1]
members: [m2 m3]
```

currently active members: [m2 m3]

Now lets add m1 back in:

```
cg elastic add ORDERS_DYNAMIC d-consumer-grp m1
```

```
cg elastic info ORDERS_DYNAMIC d-consumer-grp
```

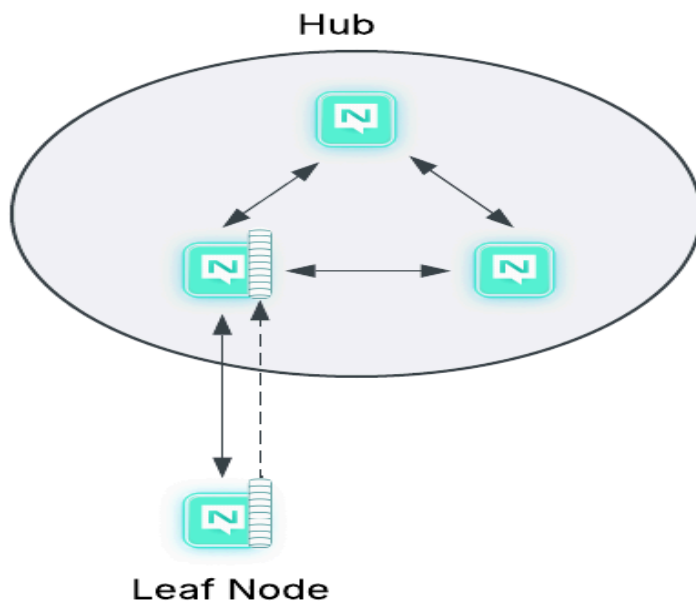
You should now see m1 one consuming messages again !

So as you can see with elastic mode:

- You can make workers step down so the standby takes over
- You can add and remove workers at will and the work will continue unhindered, as the load distribution is automatically rebalanced !

Leaf Node Topologies

In this section we will be looking at the Leaf Node topology and will explore the Store and forward model.



To that effect we will be using a single Leaf Node connected to a 3 node Hub cluster.

Hands-on exercises

To start the cluster up we run:

```
docker compose -f LeafNodes/compose-cluster-leaf.yml up
```

Once the cluster is up, we can test to see the membership:

```
nats server list -s nats://localhost:4772 --user sys --password sys  
nats server list -s nats://localhost:4775 --user sys --password sys
```

Lets create a stream on the LeafNode:

```
nats stream add IOT_LEAF_STREAM --user user_leaf --password password -s  
nats://localhost:4775 --subjects "iot.data.>" --storage file --defaults
```

Lets look at some data about the stream:

```
nats stream ls -s nats://localhost:4775 --user user_leaf --password password  
nats stream report -s nats://localhost:4775 --user user_leaf --password password
```

Now lets create a sourced stream on a Hub Node; the Source stream synchronizes live (async) with the Leaf node stream.

NOTE: in the creation of the sourced stream, we must mention LEAF_DOMAIN in the config process:

```
nats stream add NODE1_IOT_STREAM --user user_a --password password -s  
nats://localhost:4772 --storage=file --source=IOT_LEAF_STREAM --defaults
```

```
[cluster_sys] ? Adjust source "IOT_LEAF_STREAM" start Yes  
[cluster_sys] ? IOT_LEAF_STREAM Source Start Sequence 0  
[cluster_sys] ? IOT_LEAF_STREAM Source UTC Time Stamp (YYYY:MM:DD HH:MM:SS)  
[cluster_sys] ? Adjust source "IOT_LEAF_STREAM" filter and transform No  
[cluster_sys] ? Import "IOT_LEAF_STREAM" from a different JetStream domain Yes
```

*[cluster_sys] ? IOT_LEAF_STREAM Source foreign JetStream domain name
LEAF_DOMAIN*

*[cluster_sys] ? IOT_LEAF_STREAM Source foreign JetStream domain delivery prefix
Stream NODE1_IOT_STREAM was created*

*nats stream ls -s nats://localhost:4772 --user user_a --password password
nats stream report -s nats://localhost:4772 --user user_a --password password*

Now lets create a consumer on a Hub Node and one on the Leaf Node:

For the Hub Consumer deliver the message to the subject: *iot.hub.consumer*
For the Leaf Node consumer deliver the message to subject: *iot.leaf.consumer*

In both cases select all defaults.

*nats consumer add NODE1_IOT_STREAM MY_HUB_CONSUMER -s nats://localhost:4772
--user user_a --password password*

*[cluster_sys] ? Delivery target (empty for Pull Consumers) **iot.hub.consumer**
[cluster_sys] ? Delivery Queue Group
[cluster_sys] ? Start policy (all, new, last, subject, 1h, msg sequence) all
[cluster_sys] ? Acknowledgment policy none
[cluster_sys] ? Replay policy instant
[cluster_sys] ? Filter Stream by subjects (blank for all)
[cluster_sys] ? Idle Heartbeat 0s
[cluster_sys] ? Enable Flow Control, ie --flow-control No
[cluster_sys] ? Deliver headers only without bodies No*

*nats consumer add IOT_LEAF_STREAM MY_LEAF_CONSUMER -s nats://localhost:4775
--user user_leaf --password password*

*[cluster_sys] ? Delivery target (empty for Pull Consumers) **iot.leaf.consumer**
[cluster_sys] ? Delivery Queue Group
[cluster_sys] ? Start policy (all, new, last, subject, 1h, msg sequence) all
[cluster_sys] ? Acknowledgment policy none
[cluster_sys] ? Replay policy instant
[cluster_sys] ? Filter Stream by subjects (blank for all)
[cluster_sys] ? Idle Heartbeat 0s
[cluster_sys] ? Enable Flow Control, ie --flow-control No
[cluster_sys] ? Deliver headers only without bodies No
Information for Consumer IOT_LEAF_STREAM > MY_LEAF_CONSUMER created 2026-09-25
19:29:53*

```
nats consumer report IOT_LEAF_STREAM -s nats://localhost:4775 --user user_leaf --password password
```

```
nats consumer report NODE1_IOT_STREAM -s nats://localhost:4772 --user user_a --password password
```

Now lets create subscribers for these consumers:

```
nats sub iot.hub.consumer -s nats://localhost:4772 --user user_a --password password
nats sub iot.leaf.consumer -s nats://localhost:4775 --user user_leaf --password password
```

Now lets push some data through the Leaf Node Stream:

```
nats pub iot.data.test1 "message 1" --user user_leaf --password password -s
nats://localhost:4775 --count 5
```

You should now see that the messages were received on both the Leaf and Hub consumers.

```
nats stream --user user_leaf --password password -s nats://localhost:4775 ls
nats stream --user user_leaf --password password -s nats://localhost:4775 view
```

```
nats stream --user user_a --password password -s nats://localhost:4772 ls
nats stream --user user_a --password password -s nats://localhost:4772 view
```

Now let's shut down the Hub Node with the stream.

First lets find out what hub node the stream is on:

```
nats stream info -s nats://localhost:4772 --user user_a --password password
```

Cluster Information:

Name: HUB

Leader: hub-1

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
878b52c9139e	nats:latest	"/nats-server -c /ap..."	3 hours ago	Up 3 hours	
0.0.0.0:4775->4222/tcp, [::]:4775->4222/tcp		leafnodes-edge-node-1			
4b416ce841c2	nats:latest	"/nats-server -c /ap..."	3 hours ago	Up 3 hours	
0.0.0.0:4773->4222/tcp, [::]:4773->4222/tcp		leafnodes-hub-2-1			

```
086f977086b9 nats:latest "/nats-server -c /ap..." 3 hours ago Up 3 hours
0.0.0.0:4772->4222/tcp, [::]:4772->4222/tcp leafnodes-hub-1-1
026f2c4b0dc2 natsio/nats-box:latest "/entrypoint.sh tail..." 3 hours ago Up 3 hours
leafnodes-nbox-1
27a27c771778 nats:latest "/nats-server -c /ap..." 3 hours ago Up 3 hours
0.0.0.0:4774->4222/tcp, [::]:4774->4222/tcp leafnodes-hub-3-1
```

Docker stop leafnodes-hub-1-1

Now lets publish some more messages to the Leaf Node stream:

```
nats pub iot.data.test1 "message 2" --user user_leaf --password password -s
nats://localhost:4775 --count 5
```

```
nats stream --user user_leaf --password password -s nats://localhost:4775 ls
nats stream --user user_a --password password -s nats://localhost:4772 ls
```

Restart the Hub Node:

```
docker start leafnodes-hub-1-1
docker ps
nats stream --user user_a --password password -s nats://localhost:4772 ls
```

Now you should see that the Hub stream has completely synchronized with the Leaf Stream.

In summary, Leaf Nodes are a capability unique to NATS. With them, one can create event fabrics that include edge devices/networks, CCloud, and on-prem environments.